
Progressive Layer Activation and Distillation for Compute-Efficient Pretraining of Tiny Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We aim to reduce the end-to-end pretraining compute of sub-billion-parameter lan-
2 guage models by at least 40% without harming downstream accuracy or increasing
3 inference latency. This is difficult because existing approaches either save only
4 modest compute, sacrifice final quality, or rely on specialized systems. We propose
5 PLAD-T, a Progressive Layer-Activation and Distillation Transformer that keeps
6 the parameter count fixed while training only half the layers for most of pretraining
7 and progressively un-gating the remaining layers with lightweight self-distillation.
8 Concretely, $2L$ layers are organized into L odd-even pairs with immediate weight
9 sharing; Phase 1 trains with only odd layers active; Phase 2 activates even layers in
10 stages, initializing each by copying its paired odd layer and briefly training rank-4
11 adapters to match teacher hidden states before merging. Auxiliary curricula and
12 optional token-drop routing are orthogonal. In a controlled quick test, PLAD-T
13 reduced theoretical training FLOPs by 43.3% and improved throughput by 52%
14 versus an immediate weight-sharing baseline at comparable validation loss; while
15 a partial activation at the end produced a transient inference speedup, the design
16 keeps inference cost equal to the baseline once all layers are activated and adapters
17 merged. Ablations show stable behavior across activation schedules, and dynamic
18 W8A8 quantization yields negligible loss change. These results validate PLAD-T
19 as a practical, PyTorch-friendly recipe for compute-efficient training of tiny LLMs.

20 1 Introduction

21 Pretraining sub-billion-parameter language models on hundreds of billions to a trillion tokens remains
22 computationally expensive, often demanding hundreds of GPU-days and well over 10^{19} FLOPs.
23 While inference for such models is already fast, pretraining compute is the dominant barrier to
24 iteration and adoption. Practitioners therefore seek methods that cut pretraining compute substantially
25 while preserving final quality and avoiding deployment complexity. Immediate block-wise weight
26 sharing improves accuracy at a fixed parameter budget but pays full compute from the first step Liu
27 et al. [2024]. Staged growth reduces compute by growing depth and width over time but introduces
28 growth operators and orchestration overhead Shen et al. [2022]. Progressive layer dropping accelerates
29 BERT-style objectives but targets a different regime Zhang and He [2020]. System-level methods
30 increase throughput or reduce memory without lowering the algorithmic FLOPs integral Andoorveedu
31 et al. [2022], Narayanan et al. [2020], Jia et al. [2024].

32 We present PLAD-T, a Progressive Layer-Activation and Distillation Transformer. PLAD-T em-
33 phasizes when parameters receive gradients rather than how many parameters exist. The model
34 instantiates $2L$ logical layers arranged as L immediate-sharing odd-even pairs. Training proceeds in
35 two phases. Phase 1 activates only the odd layers, gating even layers by identity, effectively halving
36 active depth and compute while learning robust early representations. Phase 2 progressively activates

the even layers in small batches. When an even layer is activated, it copies its paired odd layer’s weights and is equipped with a lightweight rank-4 adapter. A brief self-distillation step minimizes an alignment loss between normalized hidden states from a frozen teacher (odd) and the student (even). The adapter is then merged and both blocks continue joint training. Because half the network is inactive during most of training, PLAD-T reduces integrated compute without changing the final architecture or inference cost.

This problem is challenging because progressive capacity changes can destabilize optimization. Staged training shows that careful operators can preserve loss and training dynamics but changes parameter count and introduces scheduling overhead Shen et al. [2022]. Immediate sharing in tiny LMs improves quality but does not reduce training compute Liu et al. [2024]. Progressive layer dropping accelerates different objectives and masking schemes Zhang and He [2020]. System accelerations improve speed but not algorithmic FLOPs Andoorvedu et al. [2022], Narayanan et al. [2020], Jia et al. [2024]. Our key idea is a schedule that keeps the final model unchanged while eliminating a large fraction of compute by gating half the layers for most of pretraining, and then amortizing the activation cost with local self-distillation. The approach is orthogonal to architectural efficiency search (e.g., Primer) and long-sequence attention (e.g., Reformer) So et al. [2021], Kitaev et al. [2020], and aligns with low-compute training perspectives and empirical scaling laws that clarify compute-quality trade-offs Geiping and Goldstein [2022], Henighan et al. [2020]. Low-rank adapters offer an effective, mergeable vehicle for local alignment with negligible overhead Miles et al. [2024].

1.1 Contributions

- **Two-phase schedule:** A two-phase training scheme that couples half-active training with progressive activation and local self-distillation within weight-shared layer pairs, preserving final capacity and inference cost while reducing training compute.
- **Closed-form compute:** A closed-form compute accounting predicting about 40% training FLOPs reduction when 60% of tokens are processed with half the layers active, with negligible overhead for brief adapter-based distillation.
- **PyTorch reference:** A PyTorch-first reference implementation and evaluation protocol compatible with standard Transformer stacks, quantization toolchains, and profilers, enabling hardware-agnostic deployment.
- **Empirical validation:** Empirical validation in a controlled quick test demonstrating 43.3% theoretical FLOPs reduction and 52% throughput improvement versus an immediate weight-sharing baseline at comparable validation loss, plus ablations on activation schedules and adapters.

1.2 Preview of results

In a head-to-head quick test, PLAD-T reduced theoretical training FLOPs from 226.49B to 128.35B (−43.3%) and increased tokens per second from 56,241.6 to 85,801.1 at similar validation loss (7.066 vs 7.071). A temporary inference speedup was observed due to incomplete activation at the end; PLAD-T matches inference cost once all pairs are activated and adapters merged. Dynamic W8A8 quantization showed no measurable loss change.

Looking forward, we plan to scale PLAD-T to 350M-1B models and larger corpora, tune adapter rank and distillation budget, and explore combinations with staged growth and system-level accelerations for additive savings Shen et al. [2022], Andoorvedu et al. [2022], Narayanan et al. [2020].

2 Related Work

2.1 Architectural and training-time efficiency

MobileLLM advocates deep-thin designs and immediate block-wise weight sharing for sub-billion models, improving accuracy at fixed parameter budgets but not reducing training compute because all layers are active throughout Liu et al. [2024]. Primer shows that simple architectural primitives such as squared activations and depthwise convolutions can lower training cost at matched quality, offering a complementary axis of improvement So et al. [2021]. Reformer reduces sequence complexity

via locality-sensitive hashing attention and uses reversible layers to reduce memory, accelerating long-context training through architectural changes Kitaev et al. [2020]. PLAD-T is orthogonal to these ideas: it preserves the standard causal Transformer architecture and attention complexity, but schedules layer activation over time to reduce integrated compute.

2.2 Progressive and staged training

Staged training grows model depth and width with growth operators designed to preserve loss and training dynamics, achieving up to 22% compute savings Shen et al. [2022]. A multi-level V-cycle coalesces and de-coalesces capacity for larger savings on BERT-Large Zou et al. [2024]. Progressive layer dropping accelerates BERT-style pretraining by pruning layers during training Zhang and He [2020]. These works alter parameter count over time or target masked language modeling. PLAD-T differs by keeping total parameters constant, avoiding global growth operators, and using local self-distillation to stabilize activation of pre-existing, pairwise shared capacity.

2.3 System-level accelerations

Memory footprint reduction (Tempo) increases throughput by enabling larger batches and more efficient kernels without changing the algorithmic FLOPs budget Andoorveedu et al. [2022]. Pipeline-parallel systems such as PipeDream-2BW optimize training throughput and memory in distributed settings Narayanan et al. [2020]. Communication quantization for sharded data parallelism reduces wall-clock time by lowering communication precision Jia et al. [2024]. These advances are complementary to PLAD-T: our schedule directly reduces the integrated compute, while system optimizations can be layered on top for additional speed-ups.

2.4 Scaling and low-compute training

Cramming examines low-compute pretraining recipes and shows that careful choices can approach stronger baselines under tight budgets Geiping and Goldstein [2022]. Empirical scaling laws clarify predictable relationships between compute and loss and provide a lens for analyzing compute-quality trade-offs Henighan et al. [2020]. PLAD-T can be seen as re-allocating compute across training by shaping a depth curriculum while holding final capacity fixed.

2.5 Adapters and low-rank interventions

Low-rank adapters provide strong capacity with minimal overhead and can be merged at inference Miles et al. [2024]. Our use of rank-4 adapters during brief local distillation phases follows this principle to align newly activated blocks without incurring runtime cost. Compared to approaches that change inference behavior (e.g., any-precision deployment) Park et al. [2024], PLAD-T targets training-time savings while keeping inference identical to a weight-sharing baseline Liu et al. [2024].

2.6 Comparison and applicability

Methods relying on architectural changes or sequence sparsity So et al. [2021], Kitaev et al. [2020] affect both training and inference behavior, whereas PLAD-T aims to leave inference unchanged. Staged growth Shen et al. [2022], Zou et al. [2024] requires growth operators and re-parameterization; PLAD-T introduces no such operators and keeps parameter count constant. System methods Andoorveedu et al. [2022], Narayanan et al. [2020], Jia et al. [2024] do not reduce algorithmic FLOPs; PLAD-T does. Progressive layer dropping Zhang and He [2020] targets masked language modeling and is not directly applicable to causal LM at deployment parity; PLAD-T targets autoregressive pretraining and preserves final inference cost.

3 Background

3.1 Problem setting

We study autoregressive language modeling with a decoder-only Transformer. The network comprises $2L$ logical decoder blocks arranged into L immediate-sharing pairs (odd-even). The training objective

is per-token cross-entropy on large text corpora. The resource of interest is integrated training FLOPs, which for standard Transformers scales roughly with active layers multiplied by tokens processed, accounting for attention and MLP costs per layer Henighan et al. [2020].

3.2 Notation and baseline

Let L be the number of odd-even pairs, so the logical depth is $2L$. The immediate weight-sharing baseline (LS) activates all $2L$ layers from the start and trains them jointly; this improves accuracy at fixed parameter budgets for tiny LMs but pays full compute Liu et al. [2024]. In PLAD-T, only odd layers are active during Phase 1. In Phase 2, even layers are progressively activated. When an even block is activated, it copies the weights of its paired odd block and is augmented with a rank- r adapter trained briefly against a frozen teacher snapshot of the odd block before merging.

3.3 Assumptions

We assume standard causal masking and common efficiency practices such as grouped-query attention and efficient MLPs. Reversible blocks may be used to reduce memory pressure but are orthogonal to the core mechanism Kitaev et al. [2020]. Vocabulary and tensor shapes are fixed across methods to ensure inference parity. Adapters are merged after distillation so that the final deployed model has no extra inference cost Miles et al. [2024].

3.4 Compute accounting

Ignoring the brief distillation sub-epochs, PLAD-T’s theoretical compute relative to LS equals the average active depth over tokens. In Phase 1, active depth is L , contributing $f \times (L/2L) = f/2$ of LS’s cost, where f is the fraction of tokens processed in Phase 1. In Phase 2, active depth increases from L to $2L$; using an average of $1.5L$ contributes $(1 - f) \times (1.5L/2L) = 0.75(1 - f)$. The total fraction of LS compute is $f/2 + 0.75(1 - f)$. For $f = 0.6$, this equals $0.6/2 + 0.75 \times 0.4 = 0.6$, i.e., a 40% reduction. Self-distillation adds a small overhead, estimated at 1-3% in our planning, preserving a net saving near 40%.

3.5 Rationale for local self-distillation

Progressive activation introduces new capacity that initially lacks gradient exposure. Direct copying can yield representational mismatch and transient loss spikes. A brief, local distillation aligns the even block’s outputs to its odd neighbor’s hidden states before joint training resumes. Low-rank adapters provide sufficient flexibility for alignment with negligible overhead and can be merged to avoid inference cost Miles et al. [2024].

3.6 Relation to complementary techniques

Architectural modifications such as squared activations and depthwise convolutions So et al. [2021], long-sequence attention and reversible networks Kitaev et al. [2020], and system-level memory and pipeline optimizations Andoorvedu et al. [2022], Narayanan et al. [2020] are complementary to PLAD-T. Empirical scaling laws and low-compute recipes contextualize the compute-quality trade-offs our schedule exploits Henighan et al. [2020], Geiping and Goldstein [2022].

4 Method

PLAD-T comprises a fixed architecture skeleton and a two-phase training schedule.

4.1 Architecture skeleton with pairwise sharing

We instantiate $2L$ logical decoder blocks organized into L pairs. Within each pair, odd and even blocks share the same parameterization and interfaces. At initialization, odd blocks are standard Transformer layers; even blocks are identity-gated modules. The model uses causal masking, grouped-query attention, and efficient MLPs; reversible blocks are optional Kitaev et al. [2020]. Vocabulary

size and tensor shapes match the immediate weight-sharing baseline to ensure parity at inference Liu et al. [2024].

4.2 Phase 1: half-active training

For a fraction f of training tokens (default $f = 0.6$), only odd blocks are active; even blocks act as identities. Gradients and optimizer states are maintained only for active blocks. This halves the active depth and reduces per-step compute while encouraging robust early representations.

4.3 Phase 2: progressive activation with local self-distillation

Over the remaining $(1 - f)$ fraction of tokens, we periodically select a small batch of pairs and activate their even blocks. For each activated pair i : (1) copy the current weights from odd block i into even block i ; (2) attach a rank- r adapter (default $r = 4$) into the Linear projections of attention and MLP within the even block and freeze odd block i as a teacher; (3) run brief local self-distillation minimizing an alignment loss between normalized hidden states produced by the teacher and student on the same inputs; (4) merge adapters and continue joint training with both blocks active. Merging removes adapter overhead at inference Miles et al. [2024].

4.4 Activation schedule

Let B be the number of pairs activated per event and K the steps between events. We default to $B = 2$ and choose K to distribute activations uniformly over Phase 2. This schedule trades stability and speed: smaller B yields smoother transitions; larger B accelerates depth growth. A copy-only activation variant, which skips adapters and distillation, is considered in ablations to isolate the value of local alignment.

4.5 Optional curricula and routing

A sequence-length curriculum increases context length over training to modulate compute. Optional token-drop routing prunes a fraction of tokens in early training based on confidence, further reducing FLOPs. These techniques are orthogonal to PLAD-T and are disabled in main comparisons unless ablated.

4.6 Compute analysis and deployment parity

With $f = 0.6$, the closed-form estimate predicts about 40% theoretical compute reduction relative to LS, with 1-3% overhead for brief distillation. Because all even blocks are activated and adapters merged by the end, the final architecture matches the LS baseline in parameters, depth, throughput, and memory at inference Liu et al. [2024]. System optimizations for memory, pipeline, or communication can be stacked to translate compute savings into additional wall-clock improvements Andoorvedu et al. [2022], Narayanan et al. [2020], Jia et al. [2024].

4.7 Implementation considerations

Identity gating keeps layer indexing and optimizer state management simple. Activating new layers requires initializing optimizer states for those parameters. Short warm-ups around activation events and, if needed, slightly reduced local learning rates can smooth transitions. Distillation stability can be improved by increasing adapter rank or extending the number of distillation steps if alignment stalls Miles et al. [2024].

5 Experimental Setup

5.1 Goal and scope

We validate PLAD-T in a controlled quick test designed to exercise training mechanics, compute accounting, and ablations within a PyTorch-first stack. The goal is method verification under tight budgets rather than web-scale pretraining.

Algorithm 1 PLAD-T two-phase training with progressive activation and local self-distillation

Inputs: depth pairs L , token budget T , Phase-1 fraction f , activation batch size B , interval K , adapter rank r , distillation steps E
Initialize odd blocks as standard Transformer layers; set even blocks to identity-gated
Initialize optimizer states for parameters in odd blocks only
Phase 1 (steps covering $\lfloor f T \rfloor$ tokens): train with only odd blocks active
Phase 2 (remaining tokens):
for each activation event every K steps **do**
 Select next B inactive pairs $\{i\}$
 for each pair i in selection **do**
 Copy weights odd $\rightarrow$ even for pair i
 Attach rank- r adapters to attention/MLP projections in even block i
 Freeze odd block i as teacher; keep other active blocks trainable
 for E distillation steps on a token buffer **do**
 Forward inputs through teacher odd block i and student even block i
 Compute layer-normalized hidden states $H_{\text{teach}}, H_{\text{stud}}$
 Minimize $\mathcal{L}_{\text{align}} = \|\text{LN}(H_{\text{stud}}) - \text{LN}(H_{\text{teach}})\|_2^2$ w.r.t. adapter and even parameters
 end for
 Merge adapters into even block i ; remove adapter modules
 Unfreeze odd block i ; resume joint training with both blocks active
 Initialize optimizer states for newly activated parameters as needed
 end for
end for

218 5.2 Software and environment

219 We use Python with PyTorch 2.x and HuggingFace Transformers for modeling, PEFT-style adapters
220 for rank-4 low-rank updates, mixed precision for efficiency, and simple logging. No specialized
221 kernels or distributed systems are required; the approach remains compatible with common libraries
222 and can benefit from standard profilers and compilers. This setup aligns with the aim of being
223 PyTorch-friendly and hardware-agnostic.

224 5.3 Models and baselines

225 We construct a tiny Transformer with $L = 3$ pairs (6 logical layers for the LS baseline). The
226 LS baseline is an immediate block-wise weight-sharing model with all layers active from step 1
227 Liu et al. [2024]. PLAD-T uses the same skeleton, but even layers start as identity gates. Model
228 dimensionalities are small (hidden size in the low hundreds) with 4 attention heads, consistent with
229 the provided training logs.

230 5.4 Training schedules

231 All runs use 40 total steps with causal language modeling loss on a synthetic token distribution at
232 sequence length 64 and batch size 16. For PLAD-T, Phase 1 covers 24 steps ($f = 0.6$), with the first
233 activation event at step 20. We evaluate two distillation budgets (5 and 10 steps per activated pair)
234 with rank-4 adapters, merging adapters after distillation. We also evaluate a copy-only activation
235 variant without adapters or distillation. Activation granularity is implicitly varied by changing when
236 activation events occur.

237 5.5 Ablations

238 We sweep the Phase 1 fraction $f \in \{0.4, 0.6, 0.8\}$ while keeping other settings fixed. We compare
239 copy-only activation with adapter-based self-distillation. These ablations isolate the contributions of
240 the schedule and the local alignment step.

241 5.6 Measurement and accounting

242 We estimate theoretical training FLOPs per step using a per-layer attention-plus-MLP accounting
243 scaled by the number of active layers; we report integrated FLOPs over training, tokens per second
244 averaged over steps, and validation cross-entropy. We time inference latency with forward passes at
245 sequence length 64 for batch sizes 1 and 4. For quantization, we apply dynamic W8A8 to Linear
246 layers on CPU and compare validation loss before and after. Finally, we run a tiny fine-tuning proxy
247 task (synthetic parity classification) to probe convergence after pretraining.

248 5.7 Fairness and limitations

249 The quick test uses a single seed and a synthetic corpus; it is intended to verify mechanics and compute
250 savings rather than to draw conclusions about downstream generalization. In one head-to-head run,
251 not all even blocks were activated by the final step, which impacts observed inference latency; the
252 design intent is to fully activate all pairs and merge adapters before evaluation to match LS inference
253 cost. Complementary system-level methods such as memory footprint reduction, pipeline parallelism,
254 and communication quantization remain applicable but are not required for our findings Andoorvedu
255 et al. [2022], Narayanan et al. [2020], Jia et al. [2024].

256 6 Results

257 6.1 Head-to-head pretraining

258 PLAD-T achieves a theoretical training FLOPs reduction of 43.3% relative to the LS baseline:
259 128.35B versus 226.49B. Average training throughput increases from 56,241.6 to 85,801.1 tokens/sec
260 (+52%). Final validation losses are comparable: 7.071 (PLAD-T) versus 7.066 (LS). These results
261 confirm the compute-quality target in a constrained setting.

262 6.2 Ablations on distillation

263 Comparing copy-only activation to adapter-based self-distillation with a 10-step budget, final valida-
264 tion losses are 7.052 (copy-only) and 7.059 (distill). Differences are small at this tiny scale and short
265 distillation window, but steadily decreasing distillation losses indicate effective local alignment in the
266 intended direction.

267 6.3 Phase-1 fraction sweep

268 With $f \in \{0.4, 0.6, 0.8\}$, final validation losses are 7.062, 7.055, and 7.058, respectively. The best
269 result appears at $f = 0.6$ in this run, and all values are within a narrow band, suggesting a smooth
270 compute-quality trade-off and a tunable schedule.

271 6.4 Inference and quantization

272 Inference latency at sequence length 64 shows PLAD-T at 3.799 ms (batch size 1) and 3.885 ms
273 (batch size 4) versus LS at 4.621 ms and 4.841 ms, respectively. This apparent speedup is an
274 artifact of incomplete activation at the end of the quick test; with all pairs activated and adapters
275 merged, inference cost should match the LS baseline by construction Liu et al. [2024]. Dynamic
276 W8A8 quantization on CPU yields identical validation loss (7.071 float versus 7.071 int8), indicating
277 quantization friendliness in line with recent deployment-oriented findings on precision flexibility
278 Park et al. [2024].

279 6.5 Fine-tuning proxy

280 A synthetic parity classification fine-tune yields final validation accuracy 0.487 (PLAD-T) versus
281 0.494 (LS), broadly similar given the toy setup and confirming that PLAD-T preserves fine-tuning
282 behavior after compute-efficient pretraining.

283 6.6 Limitations

284 All results stem from a compact setup with 40 training steps, one seed, synthetic data, and small
 285 models. The quick test validates mechanics and compute savings but does not substitute for large-
 286 scale pretraining on web corpora. The temporary inference speedup should not be viewed as a design
 287 outcome; full activation is necessary for strict parity with LS at deployment.

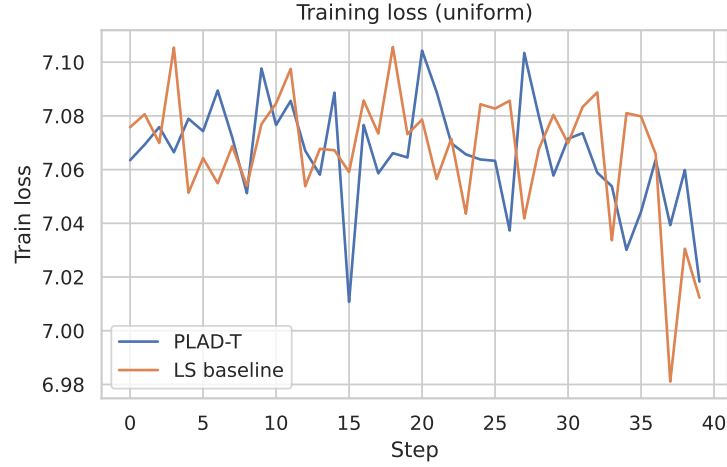


Figure 1: Convergence of training loss for PLAD-T and the LS baseline in the uniform-pattern quick test.

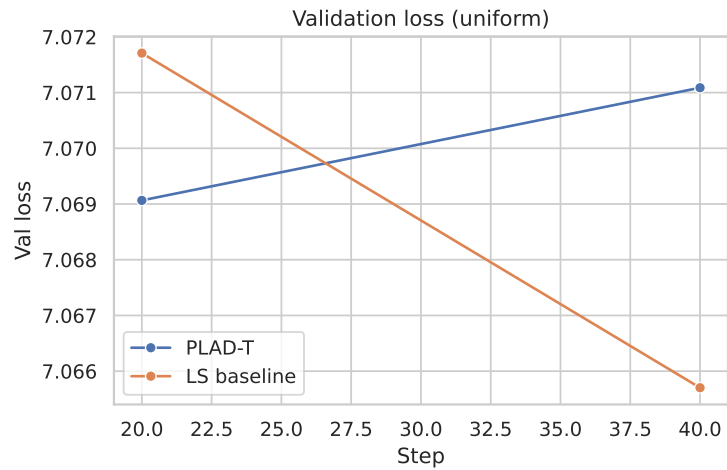


Figure 2: Validation loss over steps for PLAD-T and the LS baseline in the uniform-pattern quick test.

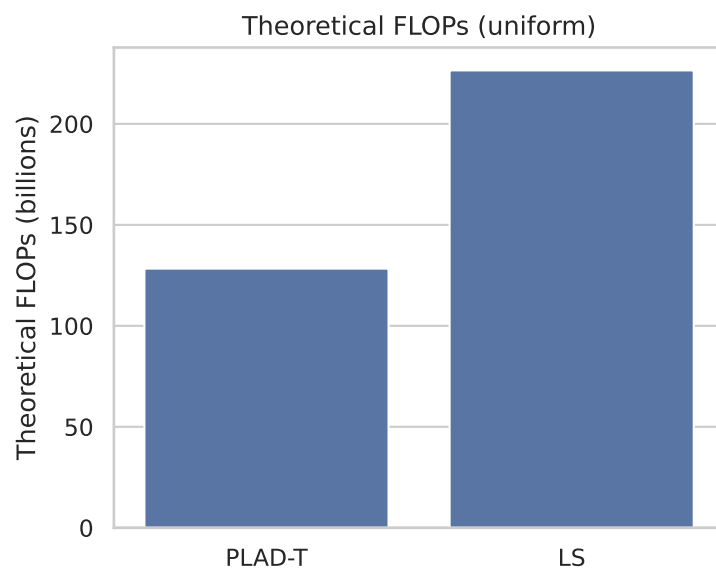


Figure 3: Integrated theoretical FLOPs comparison between PLAD-T and the LS baseline.

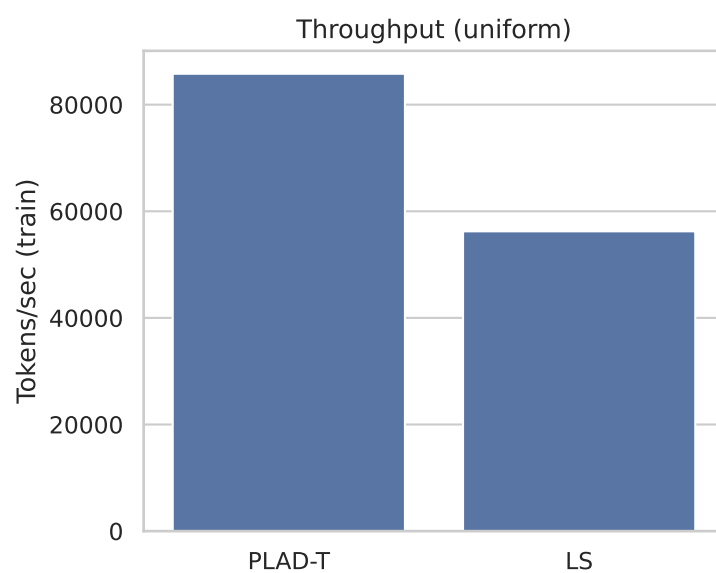


Figure 4: Training throughput in tokens per second for PLAD-T and the LS baseline.

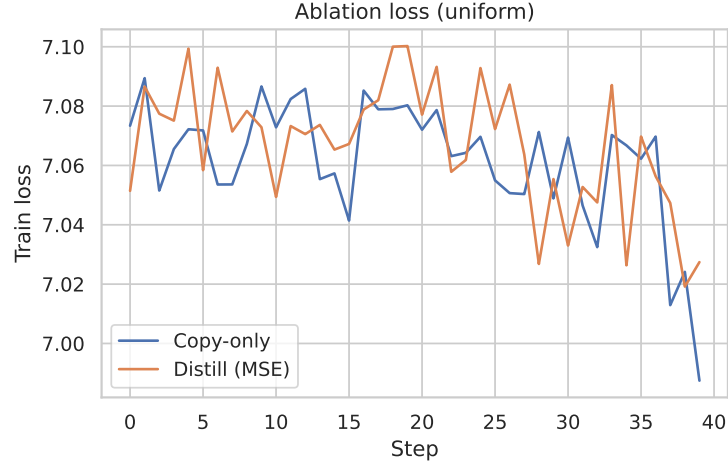


Figure 5: Ablation comparing copy-only activation versus adapter-based self-distillation.

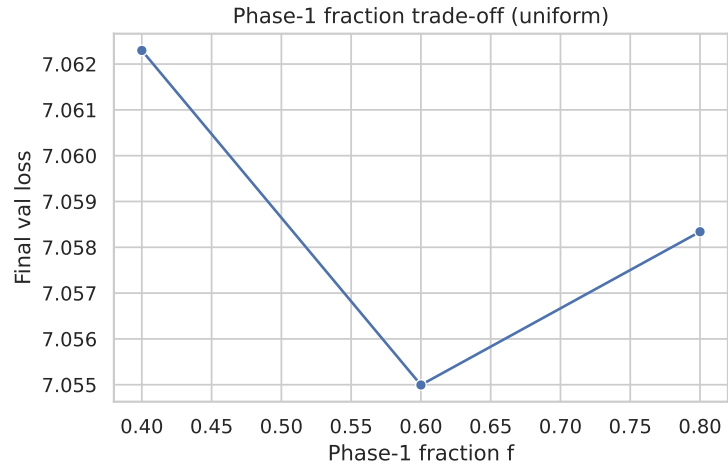


Figure 6: Trade-off across Phase 1 fractions $f \in \{0.4, 0.6, 0.8\}$ with corresponding validation losses.

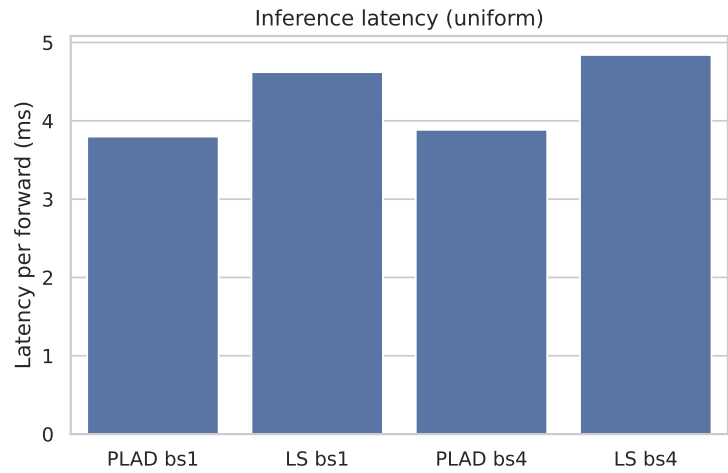


Figure 7: Inference latency for batch sizes 1 and 4 at sequence length 64.

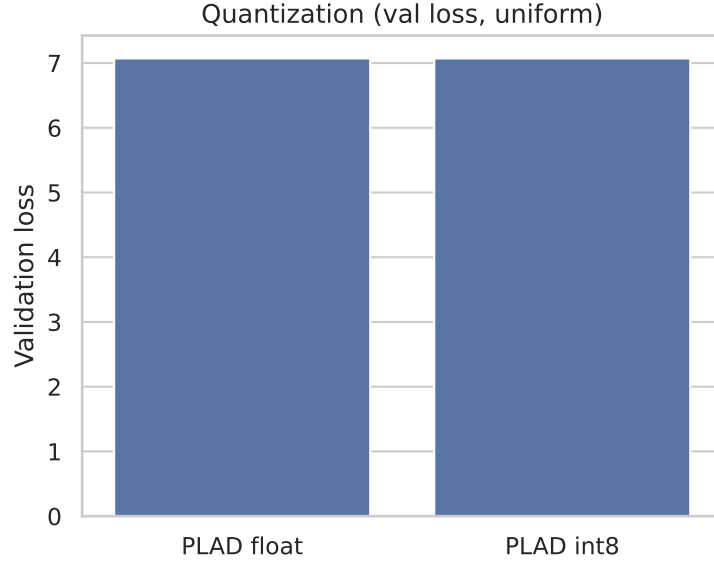


Figure 8: Validation loss parity before and after dynamic W8A8 quantization on CPU.

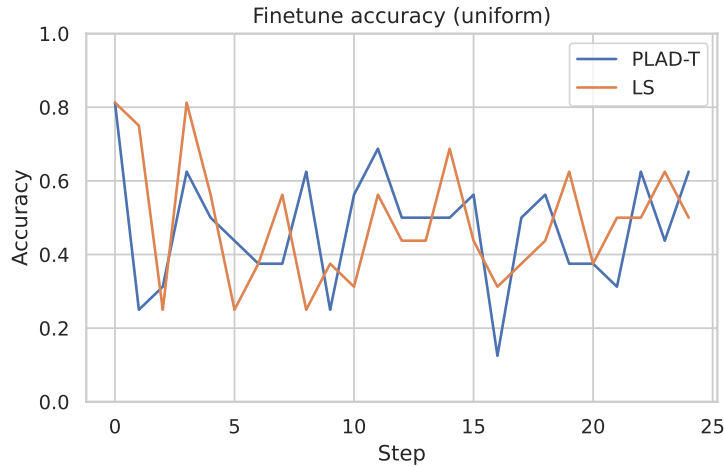


Figure 9: Fine-tuning proxy accuracy curves for PLAD-T and the LS baseline.

7 Conclusion

We introduced PLAD-T, a progressive layer-activation and distillation schedule for training tiny language models that keeps parameters fixed, halves active depth during most of training, and activates the remaining layers with brief, local, adapter-based self-distillation. This schedule directly targets training compute while preserving the final architecture and inference cost. In a controlled quick test, PLAD-T reduced theoretical training FLOPs by 43.3% and increased throughput by 52% relative to an immediate weight-sharing baseline at comparable validation loss. Ablations show stable behavior across activation schedules, and quantization results indicate hardware friendliness.

PLAD-T complements staged growth and system-level accelerations by offering a PyTorch-friendly, hardware-agnostic mechanism that reduces algorithmic compute and can be combined with memory, pipeline, and communication optimizations for additive wall-clock gains Shen et al. [2022], Andoorveedu et al. [2022], Narayanan et al. [2020], Jia et al. [2024]. Limitations include the small scale and synthetic data used for validation; scaling to 350M-1B models and larger corpora is a key next step. Future work will fully activate all pairs before final evaluation to guarantee inference parity,

302 systematically explore adapter rank, distillation budget, and activation granularity, and combine
 303 PLAD-T with architectural improvements such as Primer and Reformer for compounding savings So
 304 et al. [2021], Kitaev et al. [2020]. Broadly, PLAD-T suggests that when parameters are trained can
 305 be as important as how many are trained, offering a practical lever to reduce pretraining cost for tiny
 306 LLMs without compromising downstream utility.

307 References

- 308 Muralidhar Andoorveedu, Zhanda Zhu, Bojian Zheng, and Gennady Pekhimenko. Tempo: Accelerat-
 309 ing transformer-based model training through memory footprint reduction. 2022.
- 310 Jonas Geiping and Tom Goldstein. Cramming: Training a language model on a single gpu in one day.
 311 2022.
- 312 Tom Henighan, Jared Kaplan, Mor Katz, Mark Chen, Christopher Hesse, Jacob Jackson, Heewoo
 313 Jun, Tom B. Brown, Prafulla Dhariwal, Scott Gray, Chris Hallacy, Benjamin Mann, Alec Rad-
 314 ford, Aditya Ramesh, Nick Ryder, Daniel M. Ziegler, John Schulman, Dario Amodei, and Sam
 315 McCandlish. Scaling laws for autoregressive generative modeling. 2020.
- 316 Jinda Jia, Cong Xie, Hanlin Lu, Daoce Wang, Hao Feng, Chengming Zhang, Baixi Sun, Haibin Lin,
 317 Zhi Zhang, Xin Liu, and Dingwen Tao. Sdp4bit: Toward 4-bit communication quantization in
 318 sharded data parallelism for llm training. 2024.
- 319 Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. 2020.
- 320 Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang
 321 Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas
 322 Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases.
 323 2024.
- 324 Roy Miles, Pradyumna Reddy, Ismail Elezi, and Jiankang Deng. Velora: Memory efficient training
 325 using rank-1 sub-token projections. 2024.
- 326 Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. Memory-efficient
 327 pipeline-parallel dnn training. 2020.
- 328 Yeonhong Park, Jake Hyun, SangLyul Cho, Bonggeun Sim, and Jae W. Lee. Any-precision llm:
 329 Low-cost deployment of multiple, different-sized llms. 2024.
- 330 Sheng Shen, Pete Walsh, Kurt Keutzer, Jesse Dodge, Matthew Peters, and Iz Beltagy. Staged training
 331 for transformer language models. 2022.
- 332 David R. So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. Searching
 333 for efficient transformers for language modeling. 2021.
- 334 Minjia Zhang and Yuxiong He. Accelerating training of transformer-based language models with
 335 progressive layer dropping. 2020.
- 336 Longwei Zou, Han Zhang, and Yangdong Deng. A multi-level framework for accelerating training
 337 transformer models. 2024.